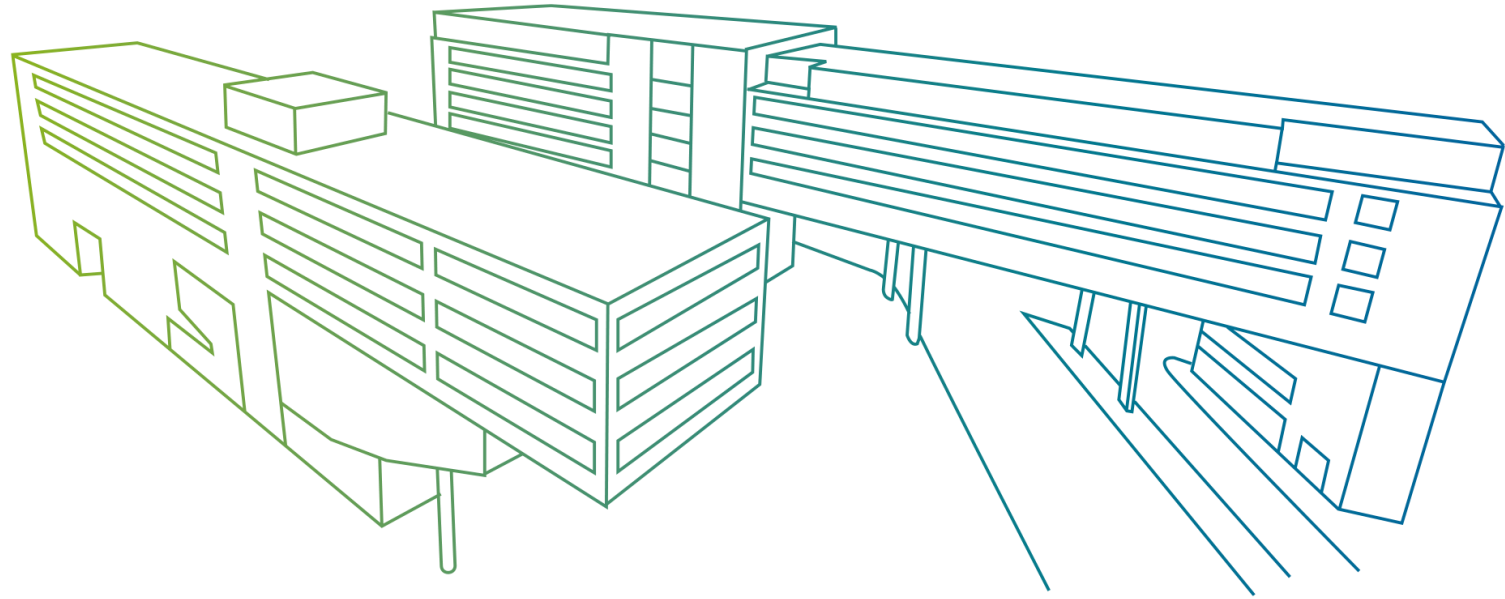




Probabilistic Robotics

Particle Filter
Lucas Muster, MSc

Particle Filters: Concept and Idea

- Our base so far:
 - Bayes Filter
 - Gaussian distributions: Process and measurement noise
 - Solving the drift problem: Use maps

→ However, we still have a single guess

→ The noise is not distributed normally

Do a simple experiment: Robot navigates N times the same path

The point cloud has a non-elliptic shape (Not a Gaussian)

Particle Filters: Multiple Guesses

- Previous algorithms used unimodal models $\{\mu_t, \Sigma_t\}$
- Representation is limited
- Multiple hypothesis are not possible

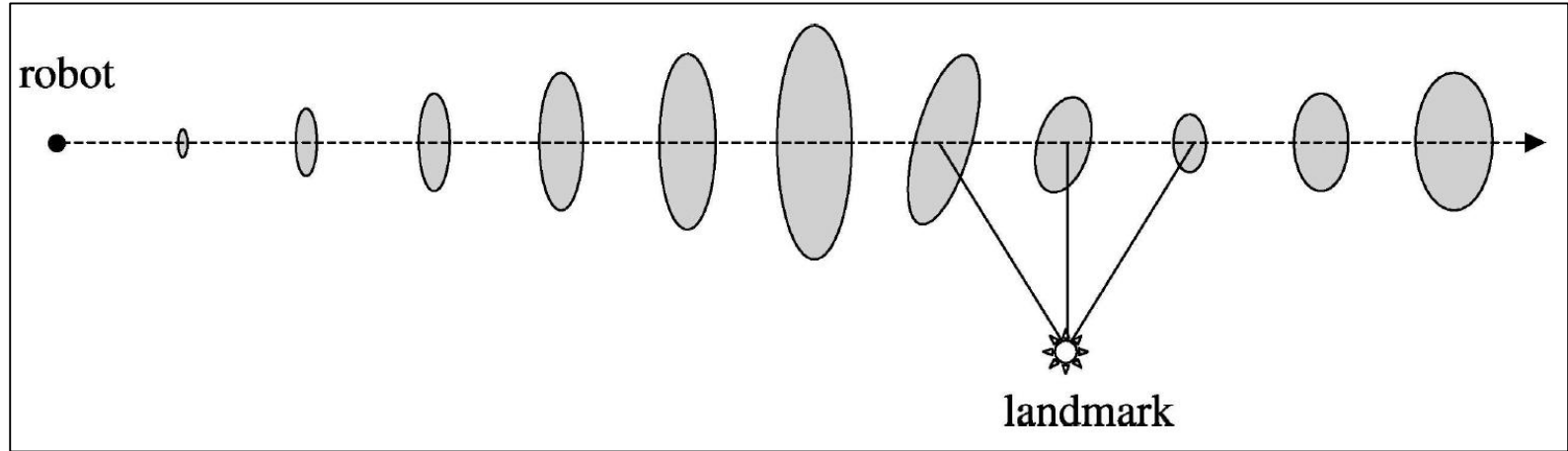
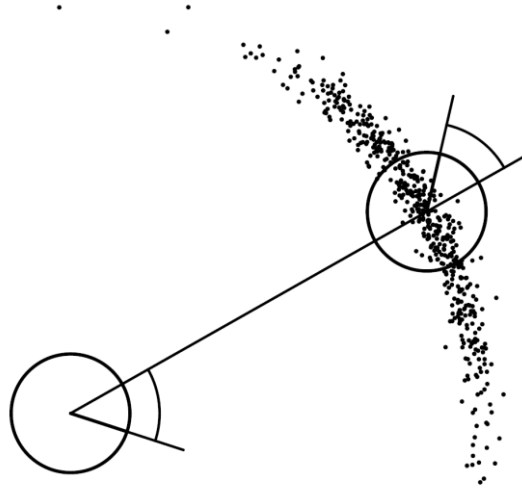


Figure: EKF Localization (Thrun, 2006). Note, that a single guess is used

Particle Filters: Concept and Idea

- Do a simple experiment: Robot navigates N times the same path



- The point cloud has a non-elliptic shape (Not a Gaussian)

Particle Filters: Real kinematic models

- So far, linear models (KF) or linearization (EKF)
- We can simulate a noisy motion:

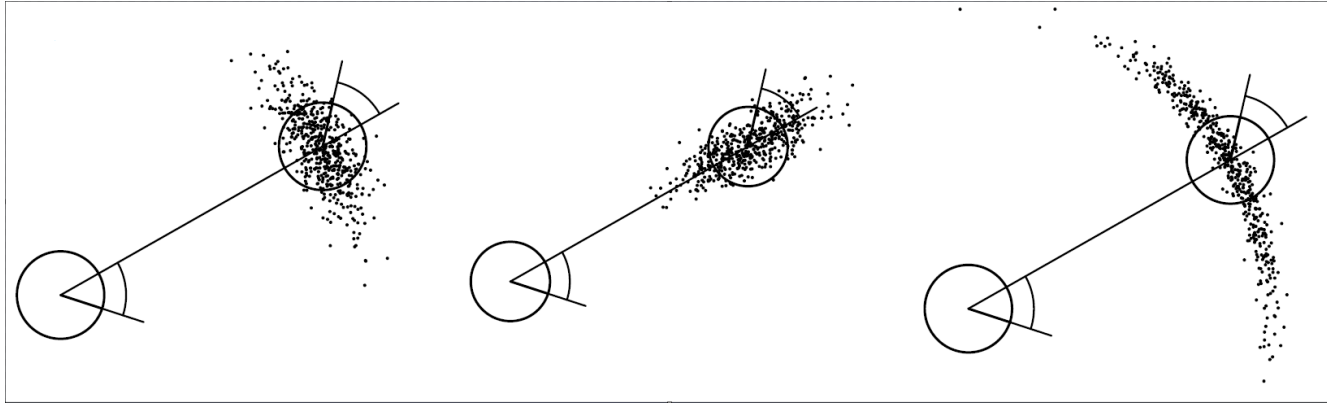


Figure: Motion simulation with different noise source (Thrun,2006)

- As you can see above, we actually do not know the source of the noise. We do have a physical model and can apply "noise" to the inputs (wheel motion).

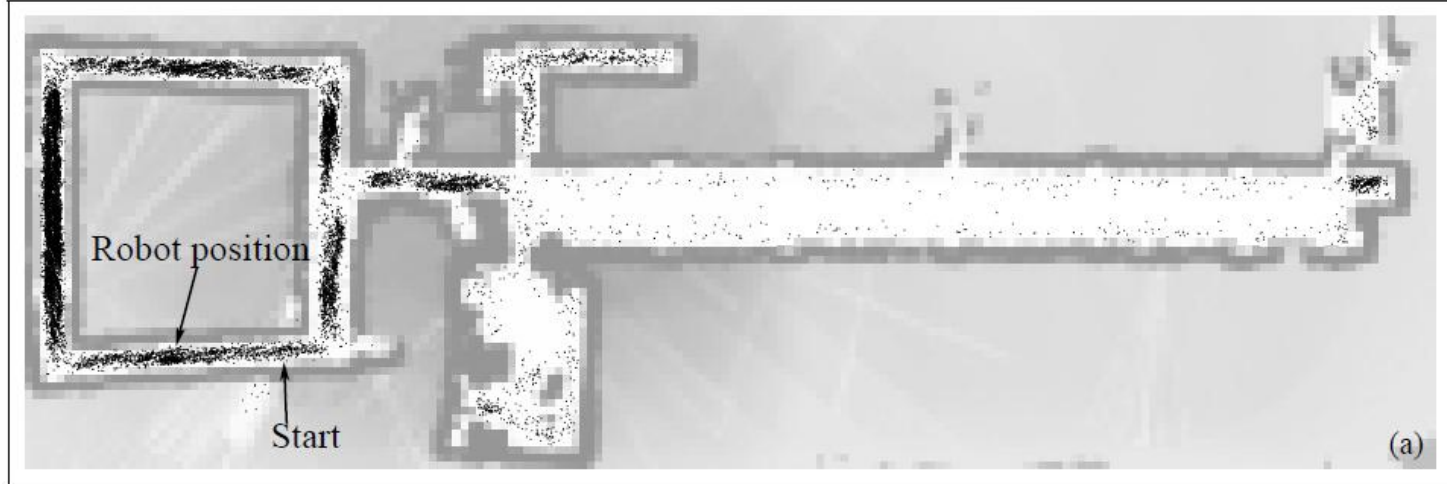
Particle Filters: Unknown Models

- Transition and Measurement models are approximations
- Error in models: Limited opportunities of "methods" (e.g.: tailor approximation)
- Model representation does not represent "real" relationships
- Goal of particle filters (in this context)
 - Represent pose of robot using position-probability on "many" possible places
 - Overcome limitations of linear and linearized models: close to reality

Particle Filters: Example

In this example, hundreds of particles were placed randomly in the map. After a few meters, the algorithm seems to start converging to the true pose, there are just a few particles in the right rooms.

However, still many particles are totally on the wrong place (see "Robot position").



Question: Why do you think do the particles vanish in the right side of the map?

Particle Filters: Example

After motion, four clusters were established.

Question 1: Why are so many particles in so few areas of the map?

Question 2: Why are four clusters at the four corners of the left side of the map?

You see, there are many hypothesis (=multiple guesses). In real applications, this approach lead to very reliable localization approaches. If a new measurement indicates, that the true pose in on the right side a lot of new particles will appear there.

To reach areas where are no particles any more, random particle placement after n steps is possible.



Particle Filters: Example

We navigate to the hallway. There is just a small cluster.

Question: Why are there no more four clusters?



Particle Filters: MCMC

- We will continue with MCMC
 - Concept
 - Sampling and re-sampling (you already know sampling)
- MCMC algorithm for mobile robots
 - Sampling from a motion model
 - Localization algorithm and map usage

Note: We already know occupancy maps and sampling from motion models

Particle Filters: Concept

- Benefit 1: Model more accurate (non-linear) coherences
- Benefit 2: Simple algorithms
- Benefit 3: Easy to understand
- Shortcoming: Complex math
 - Sampling: Take examples from distributions
 - Sequential Markov Chain Monte Carlo (MCMC) algorithms
 - Adaption for robotics (simplification)

Particle Filters: Sampling approach

- Take examples from known distribution
- We wish to calculate the expectation of a function $E[f]$

$$E[f] = \int f(\vec{z})p(\vec{z})d\vec{z}$$

- Assumption: We cannot calculate this equation (too complex, distribution unknown)
- We can take samples from $p(\vec{z})$: Using estimator $\hat{f}$
- For large number of samples: $E[f] = E[\hat{f}]$

Particle Filters: Sampling approach Basics

- Problem: We do not know $p(\vec{z})$.
- Key idea: We use a "proposal" distribution $q(\vec{z})$
- We define $q(\vec{z})$ (e.g.: Gaussian or uniform distribution)
- It is easy to sample from $q(\vec{z})$
- Due to that we can use a Monte-Carlo-Estimation:

$$E[f] = \int f(\vec{z})p(\vec{z})d\vec{z} = \int f(\vec{z})\frac{p(\vec{z})}{q(\vec{z})}q(\vec{z}) d\vec{z} \approx \frac{1}{L}\sum_{l=0}^L \frac{p(\vec{z}^l)}{q(\vec{z}^l)}f(\vec{z}^l) = \frac{1}{L}\sum_{l=0}^L r_l f(\vec{z}^l)$$

- We sample from $q(z)$ and using r_l (Importance Weight) to get unknown distribution

Particle Filters for Mobile Robots

- We apply the MCMC strategy
 - We define $p(\vec{x}_t | \vec{x}_{t-1}, \vec{u}_t)$
 - We sample from the motion model
 - This "moves" each sample
 - The sensor value is used to "weight" each particle ($p(\vec{z}_t | \vec{x}_t)$)
 - We have a measurement value
 - Relying on a pseudo-measurement and the real measurement: can we trust the pseudo measurement (= "weight")?
 - In each step, we get a new set of particles
 - We sample from set of moved particles
 - Each particle has the probability $\propto$ weight to be chosen

Particle Filters for Mobile Robots

- Initial guess using set of particles: $\chi_t = \{\vec{x}_t^{[1]}, \dots, \vec{x}_t^{[M]}\}$
- We use "particles" to
 - estimate $\overline{bel}(\vec{x}_t)$ by sampling from $p(x_t|u_t, x_{t-1})$
 - estimate $bel(\vec{x}_t)$ using re-sampling
 - Calculate weight factor $w_t^{[m]}$ for each particle using $p(\vec{z}_t|\vec{x}_t^{[m]})$
 - Generate new set of particles based on weights
- Re-sampling is reason for success of MCL

Particle Filter: Re-Sampling

- Step 1: Get $\overline{bel}(\vec{x}_t)$
 - "Move" each particle using motion model
 - Get "weight" for each moved particle using measurement model
 - "Moved" set of particles: $\bar{\chi}_t$
- Step 2: Get $bel(\vec{x}_t)$
 - Draw M particles from set $\bar{\chi}_t$
 - Draw with replacement
 - Draw probability $\propto$ weight
- Procedure is known as re-sampling or importance sampling

Particle Filter: The Algorithm

- Extension of Bayes filter
- Can approximate almost any distribution
- Easy to implement
- Accuracy is based on number of particles
- Again: Algorithm is a concept
 - Missing motion model
 - Missing measurement model
- Next step: Implementation for mobile robot

Particle Filter: The Algorithm

- Line 3-7: Implements step 1 on slide 54.
- Line 8-11: Implements step 2 of slide 16. Note: This algorithm works for localization with and without a map (dead reckoning localization).

```
1:  Algorithm Particle_filter( $\mathcal{X}_{t-1}, u_t, z_t$ ):  
2:     $\bar{\mathcal{X}}_t = \mathcal{X}_t = \emptyset$   
3:    for  $m = 1$  to  $M$  do  
4:      sample  $x_t^{[m]} \sim p(x_t \mid u_t, x_{t-1}^{[m]})$   
5:       $w_t^{[m]} = p(z_t \mid x_t^{[m]})$   
6:       $\bar{\mathcal{X}}_t = \bar{\mathcal{X}}_t + \langle x_t^{[m]}, w_t^{[m]} \rangle$   
7:    endfor  
8:    for  $m = 1$  to  $M$  do  
9:      draw  $i$  with probability  $\propto w_t^{[i]}$   
10:     add  $x_t^{[i]}$  to  $\mathcal{X}_t$   
11:    endfor  
12:    return  $\mathcal{X}_t$ 
```

Particle Filter: The Algorithm

- Need to implement motion and measurement model
- Motion model
 - Need to sample from $p(\vec{x}_t | \vec{x}_{t-1}, \vec{u}_t)$
 - Many implementations
 - Simple implementation: Using odometry and add random noise
- Measurement model
 - Need to evaluate particle using $p(\vec{z}_t | \vec{x}_t)$
 - Evaluation is weight of particle
 - Store weight and moved particle

Particle Filter: Motion Models

- We need algorithm to sample motion
- Easiest approach (see figure)
 - Using odometry
 - Add random Gaussian noise ($\mu_N = 0$ and σ_T is a tuning factor)

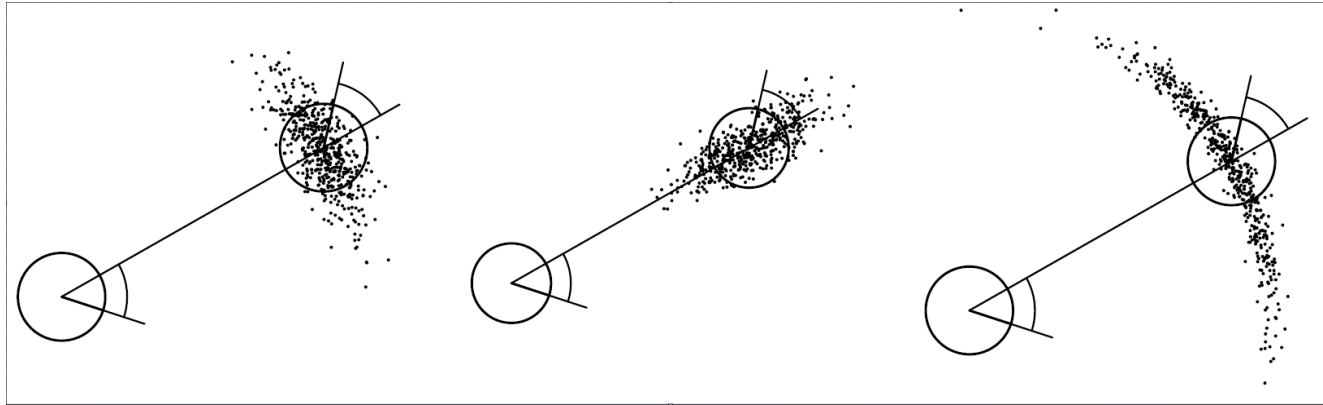


Figure: Motion sampling using three different noise settings (Thrun, 2006). Note, that there is noise for translation and rotation.

Particle Filter: Motion Models

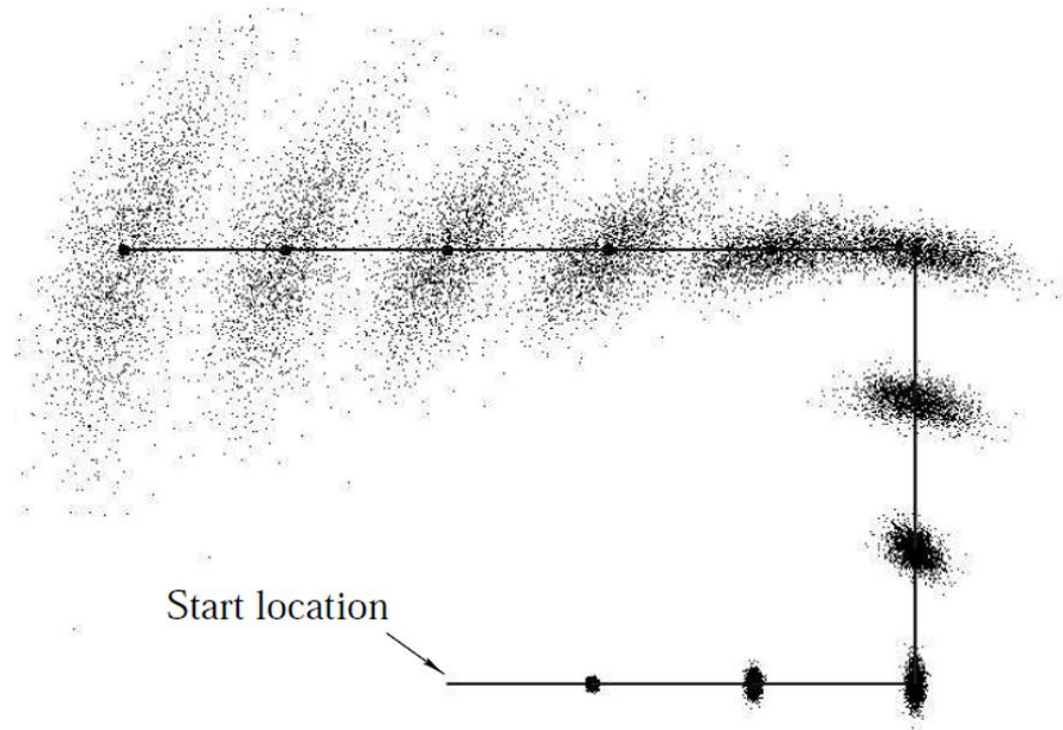


Figure: Motion model in application (Thrun, 2006). Note, that the solid line represents the actual movement.

Particle Filter: Final algorithm

```
1:   Algorithm MCL( $\mathcal{X}_{t-1}, u_t, z_t, m$ ):  
2:      $\bar{\mathcal{X}}_t = \mathcal{X}_t = \emptyset$   
3:     for  $m = 1$  to  $M$  do  
4:        $x_t^{[m]} = \text{sample\_motion\_model}(u_t, x_{t-1}^{[m]})$   
5:        $w_t^{[m]} = \text{measurement\_model}(z_t, x_t^{[m]}, m)$   
6:        $\bar{\mathcal{X}}_t = \bar{\mathcal{X}}_t + \langle x_t^{[m]}, w_t^{[m]} \rangle$   
7:     endfor  
8:     for  $m = 1$  to  $M$  do  
9:       draw  $i$  with probability  $\propto w_t^{[i]}$   
10:      add  $x_t^{[i]}$  to  $\mathcal{X}_t$   
11:    endfor  
12:    return  $\mathcal{X}_t$ 
```

Figure: The Monte Carlo Localization algorithm (Thrun, 2006).

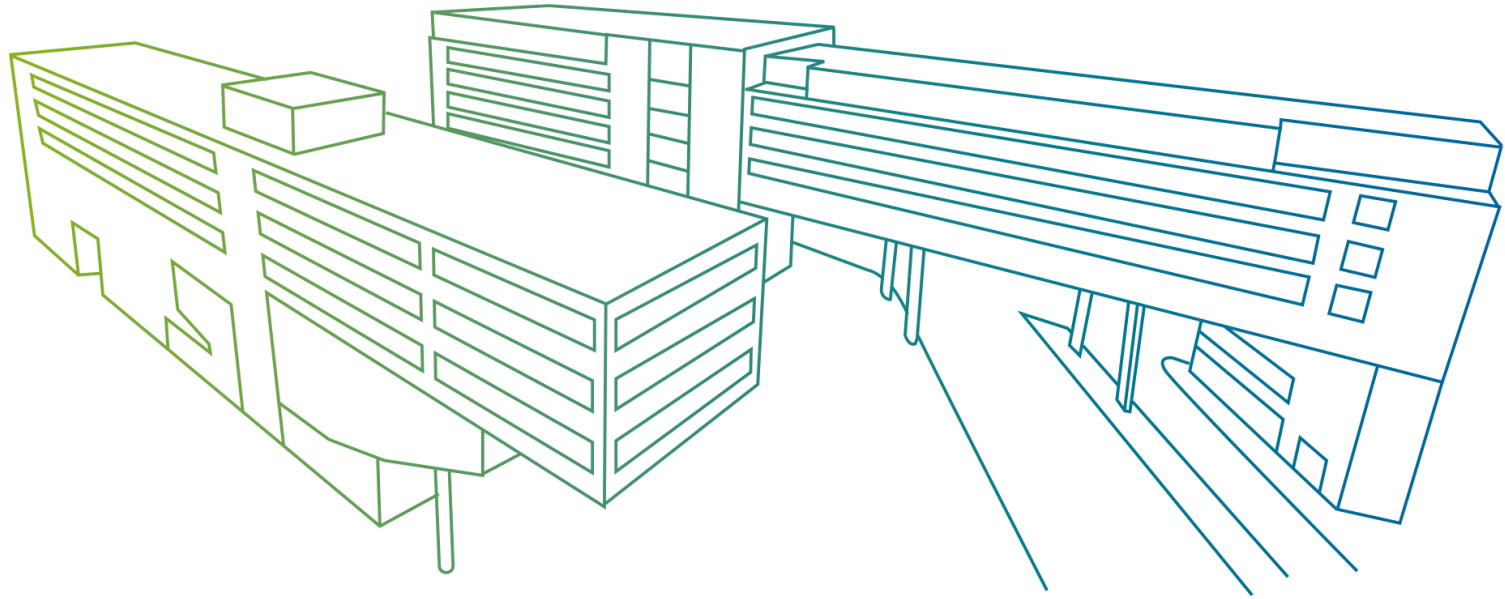
Particle Filter: Summary

- Can handle non-gaussian noise
- No restrictions concerning posterior
- This algorithm
 - cannot deal with dynamic environment
 - is restricted due to motion model (odometry and gaussian noise)
- Existing extensions to overcome those problems
- We can now recover from failures
- Alternative: Grid methods

Particle Filter: Comparison of Approaches

	EKF	MHT	Coarse (topo- logical) grid	fine (metric) grid	MCL
Measurements	landmarks	landmarks	landmarks	raw measurements	raw measurements
Measurement noise	Gaussian	Gaussian	any	any	any
Posterior	Gaussian	mixture of Gaussians	histogram	histogram	particles
Efficiency (memory)	++	++	+	–	+
Efficiency (time)	++	+	+	–	+
Ease of implementation	+	–	+	–	++
Resolution	++	++	–	+	+
Robustness	–	+	+	++	++
Global localization	no	no	yes	yes	yes

Figure: Comparison of localization algorithms (Thrun, 2006). MHT is short for multi hypothesis tracking (e.g.: a Gaussian mixture model).
Note, the coarse grid and fine grid are not discussed in this course.: The Monte Carlo Localization algorithm (Thrun, 2006).



Probabilistic Robotics

Particle Filter
Lucas Muster, MSc